

■ InventoryDAO.java Explanation (Tanglish) - Colored Version

■ Package Declaration

```
package com.inventory.dao;
```

➡■ Ithu DAO (Data Access Object) package la iruka class. Database oda direct operations (insert, update, delete, read) handle pannum.

■ Imports

➡■ com.inventory.model.Inventory – Product details model import panniruka.

➡■ java.sql.* – Database connection, PreparedStatement, SQLException, etc. use panna import panniruka.

■ Class Declaration

```
public class InventoryDAO { ... }
```

➡■ Ithu Data Access Object class – database la data store panna & fetch panna logic irukum.

■ Instance Variable

```
private Connection connection;
```

➡■ Database connection object store panna variable.

➡■ Ithu constructor la initialize aagum.

■ Constructor

```
public InventoryDAO(Connection connection) { this.connection = connection; }
```

➡■ DAO object create panna podhu connection inject pannum.

➡■ Ithu reusable database connection maintain pannum.

■ addProduct Method

```
public void addProduct(Inventory product) throws SQLException { ... }
```

➡■ Ithu main method – product details database la insert pannum.

■ Step-by-step Explanation

1■■■ SQL Query define panniruka:

```
INSERT INTO inventory (product_name, price_per_unit, quantity, total_price, status)  
VALUES (?, ?, ?, ?, ?)
```

2■■■ PreparedStatement create panniruka:

```
connection.prepareStatement(sql)
```

3■■■ Parameters set panniruka:

- stmt.setString(1, product.getProductName()) → product name insert pannum

- stmt.setDouble(2, product.getPricePerUnit()) → price per unit

- stmt.setInt(3, product.getQuantity()) → quantity

- stmt.setDouble(4, product.getPricePerUnit() * product.getQuantity()) → total price auto-calc
- stmt.setString(5, product.getStatus()) → product status

4■■■ stmt.executeUpdate() → Query execute aagum and data insert aagum.

5■■■ stmt.close() → Memory leak avoid panna statement close pannum.

6■■■ Print message: "Product added successfully!"

■ Future Scope

■ Get, Update, Delete methods later add panna plan iruku.

→■ Example: getProductByld(), updateProduct(), deleteProduct() etc.

■ Clean and simple JDBC-based DAO structure ready!